799. Champagne Tower

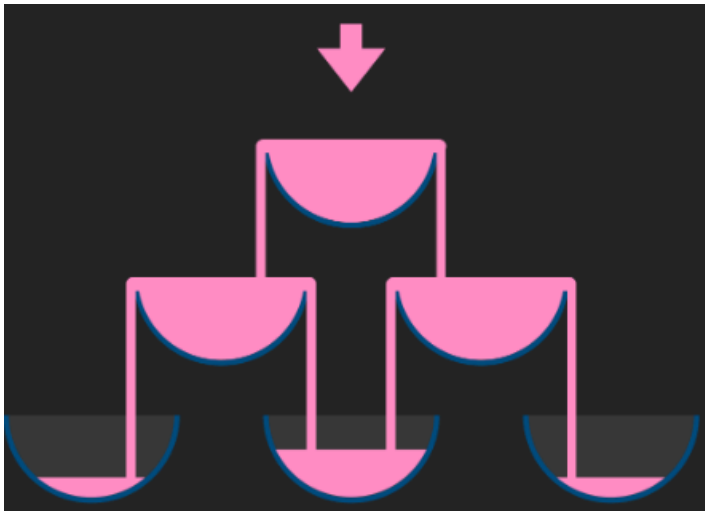
Solved

Medium | Topics | Companies

We stack glasses in a pyramid, where the **first** row has `1` glass, the **second** row has `2` glasses, and so on until the 100th row. Each glass holds one cup of champagne.

Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.)

For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.



Now after pouring some non-negative integer cups of champagne, return how full the `j^th` glass in the `i^th` row is (both `i` and `j` are 0-indexed.)

Example 1:

> **Input:** poured = 1, query_row = 1, query_glass = 1
> **Output:** 0.00000
> **Explanation:** We poured 1 cup of champange to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty.

Example 2:

> **Input:** poured = 2, query_row = 1, query_glass = 1
> **Output:** 0.50000
> **Explanation:** We poured 2 cups of champange to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champange.

Example 3:

> **Input:** poured = 100000009, query_row = 33, query_glass = 17
> **Output:** 1.00000

Constraints:

- `0 <= poured <= 10^9`
- `0 <= query_glass <= query_row < 100`

Python:

```
class Solution:
    def champagneTower(self, poured: int, query_row: int, query_glass: int) -> float:
        tower = [[0] * 102 for _ in range(102)]
        tower[0][0] = poured

        for r in range(query_row + 1):
            for c in range(r + 1):
                if tower[r][c] > 1:
                    excess = (tower[r][c] - 1.0) / 2.0
                    tower[r][c] = 1
                    tower[r+1][c] += excess
                    tower[r+1][c+1] += excess

        return tower[query_row][query_glass]
```

JavaScript:

```
var champagneTower = function(poured, query_row, query_glass) {
    const tower = new Array(query_row + 1).fill(0).map(() => new Array(query_row + 1).fill(0));
    tower[0][0] = poured;

    for (let row = 0; row < query_row; row++) {
        for (let glass = 0; glass <= row; glass++) {
            const excess = (tower[row][glass] - 1) / 2.0;
            if (excess > 0) {
                tower[row + 1][glass] += excess;
                tower[row + 1][glass + 1] += excess;
            }
        }
    }

    return Math.min(1, tower[query_row][query_glass]);
};
```

Java:

```
class Solution {
    public double champagneTower(int poured, int query_row, int query_glass) {
        double[][] tower = new double[102][102];
        tower[0][0] = (double) poured;

        for (int r = 0; r <= query_row; r++) {
            for (int c = 0; c <= r; c++) {
                if (tower[r][c] > 1.0) {
                    double excess = (tower[r][c] - 1.0) / 2.0;
                    tower[r][c] = 1.0;
                    tower[r + 1][c] += excess;
                    tower[r + 1][c + 1] += excess;
                }
            }
        }
        return tower[query_row][query_glass];
    }
}
```